

# 软工一复习

---

by zzb 2021.6.30

## 一、邵栋老师模块

---

### 1、软件开发过程

- 简称软件过程，由一组相互关联的活动组成，这些活动将一个或多个输入转化为输出，同时消耗资源来完成转换。
- **软件开发过程是指一个软件产品开发的方法，它描述了软件开发中的活动和任务。**
- “过程决定质量”
- 常见的软件开发过程模型有瀑布（线性）、迭代、和敏捷软件开发等，有很多分类的方法。
- 软件过程对软件开发具有重要的指导意义，软件开发不应该是无序的、混乱的。

原因是：软件项目通常都是很复杂的系统，开发人员需要按照一定的开发模型来分解工作任务，按照一定的步骤进行工作以完成项目的开发。如果没有开发模型的存在，开发人员很容易迷失在复杂系统中，缺乏指引，陷入混乱。

### 2、软件开发

- 软件开发是人类的一项高复杂度的**集体智力活动**，其现实问题的复杂度反映到开发中主要表现为**架构、技术、过程和组织四个维度上的复杂度**，它们往往紧密地交织纠缠在一起并相互转化。
- 软件工程发展趋势是：**架构逐渐去中心化、技术趋于自动化、过程趋于增量和迭代，以及组织趋于自治**

### 3、软件开发生命周期（SDLC）

指软件产品从**开发到报废**的生命周期，通常周期中包括了需求分析、软件设计、实现与调试、测试与验收、部署、维护等活动。**【问题定义，需求分析，软件设计，程序编码，软件测试，运行维护】**

### 4、软件开发模式

- **创建-修补**
  - 分成编码和修补两个阶段，没有长期开发计划，仅能满足当前需要，一般不会给出设计原因。
  - **优点**：适用于规模小、质量要求低的。
  - **缺点**：当添加新功能时，会因为大量的临时的局部设计而难以实现，同时，系统缺陷很多，难以修补。
- **瀑布模型**
  - 瀑布模型核心思想是根据开发活动来分解项目。
  - 如果在某个阶段出现了问题，那么最好“返回”上一个阶段并进行修改。项目开发过程像是从一个阶梯“流动”到下一个阶梯，这也是瀑布模型名称的由来。
  - 按时间顺序描述了一个软件项目的开发
  - 瀑布模型将软件**生命周期**划分为制定计划、需求分析、软件设计、程序编写、软件测试和运行维护等**六个基本活动**。
  - **优点**：为项目提供了按阶段划分的检查点；当前一阶段完成后，只需要去关注后续阶段。

- **缺点：**①通过回到前面的阶段或修改前面某个阶段的结果是非常常见的现象，而这时带来的成本增加和系统开发的混乱是很难避免的。②在项目各个阶段之间极少有反馈，开发人员很难判断其前期的工作是否正确。③**只有在项目生命周期的后期才能看到结果。**增加需求误解的可能性。

## • 迭代式软件开发

- 根据软件项目的不同功能子集来分解项目。在迭代式开发方法中，整个开发工作被组织为一系列的短小的项目，被称为一系列的迭代。每个迭代包含了需求、设计、实施（编码）、部署、测试等各种类型的开发活动，迭代完成之后需要对迭代完成的结果进行评估，并以此为依据来制定下一次迭代的目标。**每个迭代周期结束时都应该得到一个经过测试的、集成起来的基本可用的软件产品。**
- **优点：**①易于应对需求变更，可以很方便地在每一个迭代结束时修改原有需求；②提高团队士气，开发人员的士气和对项目的热情决定了软件项目的成败。每次迭代都能看到自己的成果，有利于增强信心。
- **缺点：**需求或设计中的错误往往只有到了项目后期才能够被发现；对于项目风险的控制能力较弱；软件项目常常延期完成或开发费用超出预算；项目管理人员专注于文档的完成和审核来估计项目的进展情况。

测试与集成都是非常难于估算的开发活动，不能把这样的活动放到迭代开发的最后进行，否则就无法得到迭代式开发的益处。而应该在每一次迭代都进行

## 二、编程基础 I -分解与抽象

### 1、降低复杂度

1. 分解（组合）：分解后各部分复杂度要变小，各部分**关联小，相对独立。**
2. 抽象：把具体的实现**抽象成方法和接口**，接口的复杂度要比具体的实现小。只要调用接口或方法，就能实现具体的功能，二无需书写复杂的实现过程。

### 2、程序=算法+数据结构

1. 算法是计划、过程、步骤（该如何去些写）。
2. 数据结构是操作的目标和对象（算法处理的内容）。
3. 两者相互配合才能有良好的程序。

### 3、算法建模

1. 每种语言都会提供的三种机制：
  - 基本表达式：最基本的数字运算和逻辑运算。
  - 分解（组合）的方法
  - 抽象的方法
2. 两种思路：
  - 迭代：也即循环，可以正向循环，也可以反向循环
  - 递归：也即重复调用函数，递归是重复调用直到初始状态然后再逐层返回，因此要特别注意不能忘了最初始的定义，如 $0! = 1$

- (factorial 6)
- (\* 6 (factorial 5))
- (\* 6 (\* 5 (factorial 4)))
- (\* 6 (\* 5 (\* 4 (factorial 3))))
- (\* 6 (\* 5 (\* 4 (\* 3 (factorial 2)))))
- (\* 6 (\* 5 (\* 4 (\* 3 (\* 2 (factorial 1)))))
- (\* 6 (\* 5 (\* 4 (\* 3 (\* 2 1)))))
- (\* 6 (\* 5 (\* 4 (\* 3 2))))
- (\* 6 (\* 5 (\* 4 6)))
- (\* 6 (\* 5 24))
- (\* 6 120)
- 720

- (factorial 6)
- (fact-iter 1 1 6)
- (fact-iter 1 2 6)
- (fact-iter 2 3 6)
- (fact-iter 6 4 6)
- (fact-iter 24 5 6)
- (fact-iter 120 6 6)
- (fact-iter 720 7 6)
- 720

#### 4、数据建模

- 

基于算法建模的方案：

```
print("*")  
print("#")
```

- 

基于数据建模的方案：

```
print("*#")
```

1. 基础数据类型：整数，浮点数，布尔值

2. 数据的组合：

- 数组：同一类数据的组合
  - 结构体struct：不同数据的组合
  - 对象：数据和行为的组合
3. 用日志记录数据（python）：

| 你想要执行的任务                         | 此任务最好的工具                                                                                                                    |
|----------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| 对于命令行或程序的应用，结果显示在控制台。            | <code>print()</code>                                                                                                        |
| 在对程序的普通操作发生时提交事件报告(比如：状态监控和错误调查) | <code>logging.info()</code> 函数(当有诊断目的需要详细输出信息时使用 <code>logging.debug()</code> 函数)                                           |
| 提出一个警告信息基于一个特殊的运行时事件             | <code>warnings.warn()</code> 位于代码库中，该事件是可以避免的，需要修改客户端应用以消除告警<br><br><code>logging.warning()</code> 不需要修改客户端应用，但是该事件还是需要引起关注 |
| 对一个特殊的运行时事件报告错误                  | 引发异常                                                                                                                        |
| 报告错误而不引发异常(如在长时间运行中的服务端进程的错误处理)  | <code>logging.error()</code> , <code>logging.exception()</code> 或 <code>logging.critical()</code> 分别适用于特定的错误及应用领域           |

| 级别       | 何时使用                              |
|----------|-----------------------------------|
| DEBUG    | 细节信息，仅当诊断问题时适用。                   |
| INFO     | 确认程序按预期运行                         |
| WARNING  | 表明有已经或即将发生的意外（例如：磁盘空间不足）。程序仍按预期进行 |
| ERROR    | 由于严重的问题，程序的某些功能已经不能正常执行           |
| CRITICAL | 严重的错误，表明程序已不能继续执行                 |

# 日志级别

## 三、编程基础II-可计算性

### 1、停机问题

不存在这样一个程序（算法），它能够计算任何程序（算法）在 给定 输入上是否会结束（停机）。

### 2、可计算函数不一定实际可计算（在无限的时间的储存空间的条件下可计算）

### 3、可计算模型

目标是建立一整套形式化的可计算模型

1.  $\mu$ -递归函数
2. 图灵机
3. lambda演算
4. 寄存器

## 四、lambda演算

现代数字计算机可以计算的本质上都是递归函数（图灵可计算函数）；非递归函数则是计算机不可计算的。

### 1、lambda项

1. 所有的变量都是  $\lambda$  项（名为原子）
2. 若  $M$  和  $N$  是  $\lambda$  项，那么  $(M N)$  也是  $\lambda$  项（名为应用）
3. 若  $M$  是  $\lambda$  项而  $\phi$  是一个变量，那么  $(\lambda\phi.M)$  也是  $\lambda$  项（名为抽象）

### 2、 $\alpha$ 变换

设  $\lambda\phi.M$  出现在一个  $\lambda$  项  $P$  中，且设  $\psi \notin FV(M)$ ，那么把  $\lambda\phi.M$  替换成  $\lambda\psi.[\psi/\phi] M$  的操作被称为  $P$  的  $\alpha$  变换。（即把其中的自由变量替换成不属于  $M$  的其他符号）

### 3、 $\beta$ 规约

形如  $(\lambda\phi.M) N$  的  $\lambda$  项被称为  $\beta$  可约式，对应的项  $[N/\phi] M$  则称为  $\beta$  缩减项。（也就是把  $N$  作为  $\phi$  代入到  $M$  中去）

### 4、括号的省略规则

1. 最外层的括号可以省略， $(\lambda x.x)$  可以写成  $\lambda x.$
2. 左结合的**应用型的lambda项**，如  $((((MN)P)Q))$ ，括号可以省略，表示为  $M N P Q$ （这个规则十分重要）
3. **抽象型的lambda项**， $(\lambda\phi.M)$  的  $M$  里最外层的括号可以省略，如  $\lambda x.(xy)$  可以写成  $\lambda x.xy$
4. 一个函数抽象的函数体将尽最大可能向右扩展，即： $\lambda x.M N$  代表的是一个函数抽象  $\lambda x.(M N)$  而非函数应用  $(\lambda x.M) N$ 。

省略规则的具体应用（关键是找  $\lambda x.$  的抽象， $\lambda x.$  后面必然有相关的表达式和其一起作为一个抽象）：

- SUCC ZERO

$\lambda n.\lambda f.\lambda x.f (n f x) \lambda f.\lambda x.x$

不妨先看到最右边的， $x$  旁边有个  $\lambda x.$  这两者为一个抽象，应该加括号，再左边是  $\lambda f.$ ， $(\lambda x.x)$  本身是个  $\lambda$  项，所以和  $\lambda f.$  构成了一个抽象，故  $(\lambda f.(\lambda x.x))$

再看到左边的式子， $f (n f x)$  均不为抽象，而是满足第3种情况，在在抽象种的  $M$  的括号可以省略，因此  $(f (n f x))$  要括起来，旁边还有  $\lambda x.$ 、 $\lambda f.$  和  $\lambda n.$  因此都要逐一括起来， $(\lambda n.(\lambda f.\lambda x.(f (n f x))))$ 。  
所以  $((\lambda n.(\lambda f.\lambda x.(f (n f x))))(\lambda f.(\lambda x.x)))$

### 5、具体例子

lambda演算中常用的  $\lambda$  项：（为了简便，一般用  $\lambda$  来代替  $\lambda$ ）

- ZERO =  $\lambda f.\lambda x.x$
- SUCC =  $\lambda n.\lambda f.\lambda x.f (n f x)$

- $\text{TRUE} \Leftrightarrow \lambda xy.x$   
 $\text{FALSE} \Leftrightarrow \lambda xy.y$
- $\text{PLUS} = \lambda m.\lambda n.m \text{ SUCC } n$
- $\text{MULT} = \lambda m.\lambda n.\lambda f.m (n f)$
- $\text{POW} = \lambda b.\lambda e.e b$
- $\text{PRED} = \lambda n.\lambda f.\lambda x.n (\lambda g.\lambda e.e (g f)) (\lambda u.x) (\lambda u.u)$
- $\text{SUB} = \lambda m.\lambda n.n \text{ PRED } m$

## 五、编程基础IV-函数式编程范式

### 1、消除重复：

- 抽象
- 数据过滤

### 2、高阶函数

即把函数作为参数传入函数中，所谓函数的函数。如在compare函数中，可以传入比较自己定义的比较函数，达到自己想要的比较目的。这就是高阶函数的一个例子

- Java 8 Stream API
  - `long totWaitTime = messages.stream().filter(m -> m.delay > 3000).mapToLong(m -> m.delay).sum();`

#### 传入函数

既然变量可以指向函数，函数的参数能接收变量，那么一个函数就可以接收另一个函数作为参数，这种函数就称之为高阶函数。

一个最简单的高阶函数：

```
def add(x, y, f):
    return f(x) + f(y)
```

当我们调用 `add(-5, 6, abs)` 时，参数 `x`，`y` 和 `f` 分别接收 `-5`，`6` 和 `abs`，根据函数定义，我们可以推导计算过程为：

```
x ==> -5
y ==> 6
f ==> abs
f(x) + f(y) ==> abs(-5) + abs(6) ==> 11
```

用代码验证一下：

```
>>> add(-5, 6, abs)
11
```

编写高阶函数，就是让函数的参数能够接收别的函数。

#### 小结

把函数作为参数传入，这样的函数称为高阶函数，函数式编程就是指这种高度抽象的编程范式。

### 3、函数式编程范式（如Haskell语言）

编程范式的主要作用在于提供且同时决定了程序员对于程序执行的看法。

例如，在面向对象编程中，程序员认为程序是一系列相互作用的对象，而在函数式编程中一个程序会被看作是一个无状态的函数计算的序列。将计算机运算看作是数学中函数的计算，并且避免了状态以及变量的概念。

---

#### A. Functions As First-class Citizens

函数被当作头等公民，意味着函数可以作为别的函数的参数、函数的返回值，赋值给变量或存储在数据结构中，通常以高阶函数（Higher Order Functions）的形式存在。

#### B. No Side Effects

在计算机科学中，函数副作用（Side Effect）指当调用函数时，除了返回函数值之外，还对外部作用域产生附加的影响，例如修改全局变量（函数外的变量）或修改参数。而严格的函数式编程是要求函数必须没有副作用，这意味着影响函数返回值的唯一因素就是它的参数。

由于没有副作用，所以程序支持并发执行，而不用采取锁机制，完全无需担心死锁的情况出现，但在结构化编程中是不能做到的，因为每一个函数都可能修改其外部状态，也就是产生副作用，而接下来运行的函数很可能依赖于这些被修改状态的值。

因此，函数式编程非常适合那些需要高度并行的应用程序。

#### C. No Changing-state

在函数式编程中，函数就是基础元素，可以完成几乎所有的操作，哪怕是最简单的计算，也是用函数来完成的，而我们平时在其他类型中所理解的变量（可修改，往往用来保存状态）在函数式编程中，是不可修改的，这意味着状态（State）不能保存在变量中，而事实上函数式编程是使用函数参数来保存状态，最好的例子便是递归。

---

### 4、信号流

枚举器—>过滤器—>映射—>累积器

如python中的iterator,filter,map,reduce

### 5、编译器的结构

- 编译 Compile：把整个程序源代码翻译成另外一种代码，然后等待被执行，发生在运行之前，产物是「另一份代码」。如编译成字节码
- 解释 Interpret：把程序源代码一行一行的读懂然后执行，发生在运行时，产物是「运行结果」。

结构：词法分析·语法分析·语义分析·中间代码生成·代码优化·目标代码生成

### 6、编程的算法考量

- 方法一：算法驱动：只在需要 $f(x)$ 的值的时候，通过计算得到。
- 方法二：数据驱动：先通过计算，保存所有 $x$ 对应的 $f(x)$ 值，需要计算的时候直接访问。

## 六、结构化编程 I -思想

### 1、结构化方法

#### 1. 思想

- 自顶向下逐步求精
- 算法+数据结构

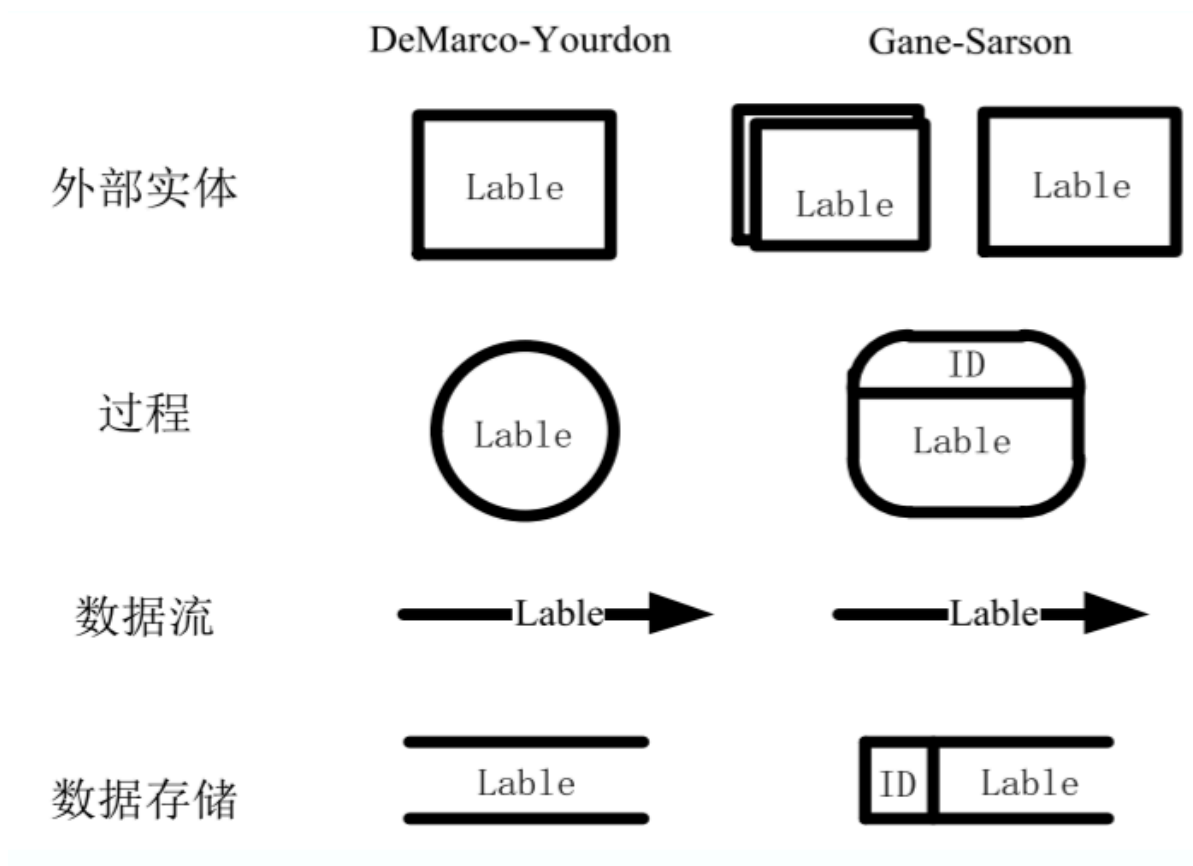
#### 2. 模型

- 数据流图
- 结构图
- 流程图

### 2、数据流图

- 将系统看做是过程的集合；
- 过程就是对数据的处理；

基本符号（一定要标准！！！第一第二种均可）

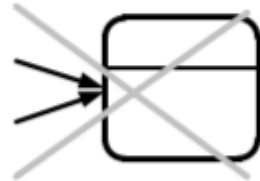


1. 外部实体：数据的生产者或者消费者
2. 过程：数据的处理转换
3. 数据流：通过过程是数据流从输入转换为输出
4. 数据存储：数据有时会被存储起来为以后使用

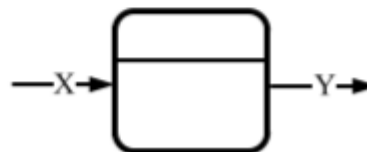
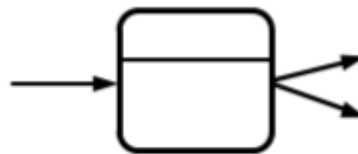
### 语法规则

1. 过程是对数据的处理，必须有输入，也必须有输出，输入数据集应该和输出数据集**存在差异**

错误的数据流



正确的数据流



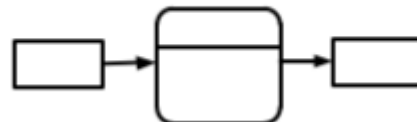
2. 数据流是必须和过程（**必须有过程参与**）产生关联的，它要么是过程的数据输入，要么是过程的数据输出

错误的数据流

数据输出



正确的数据流



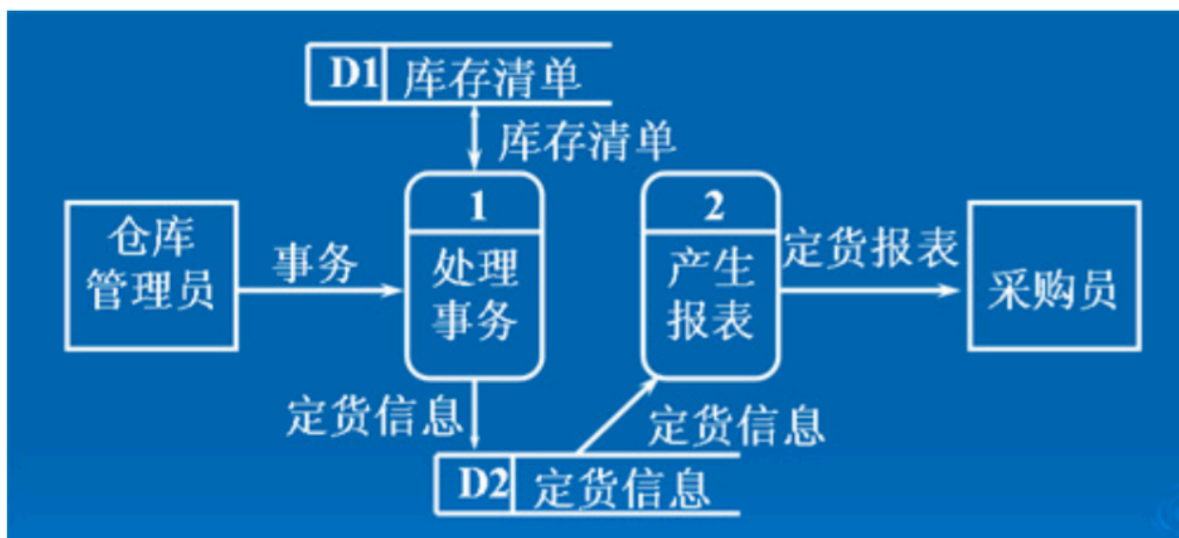
### 理解

- 过程不一定是程序，可以是任何的能够处理的模块，甚至是人工处理
- 数据存储不一定是一个文件，可能是文件的一部分，或者是数据库元素等等，它代表静态的数据
- 数据流代表动态的数据

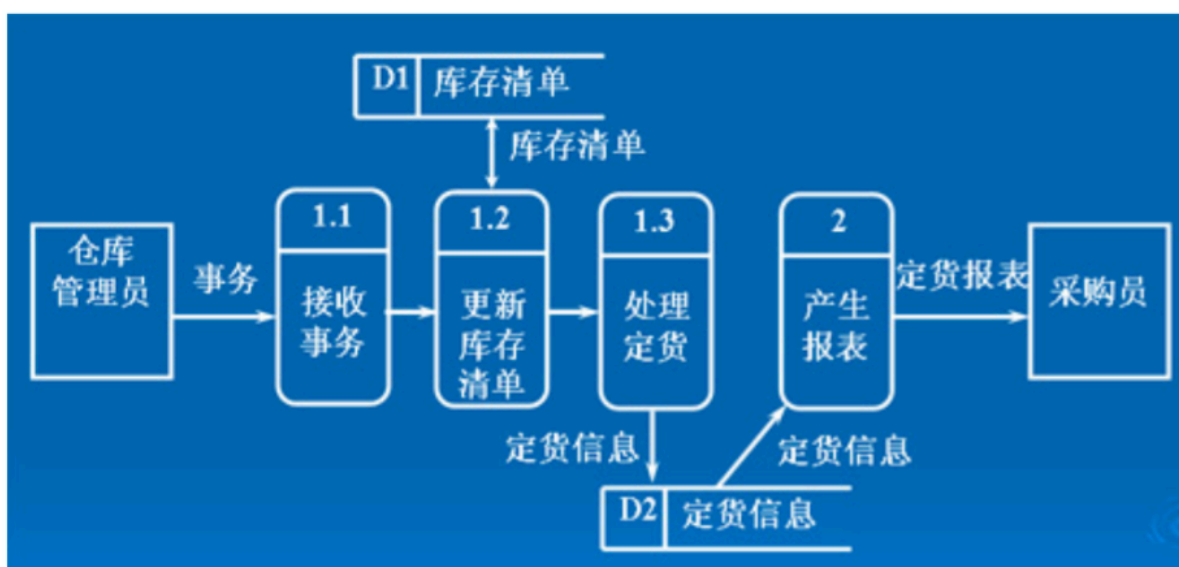
### 案例——订货系统



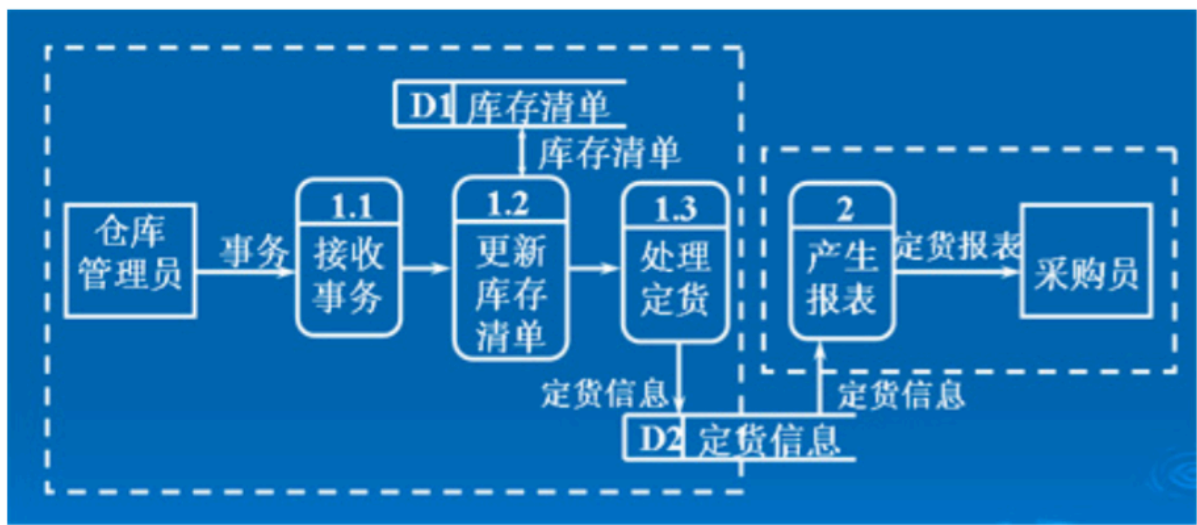
最概括的系统模型



## 细化数据存储和数据流

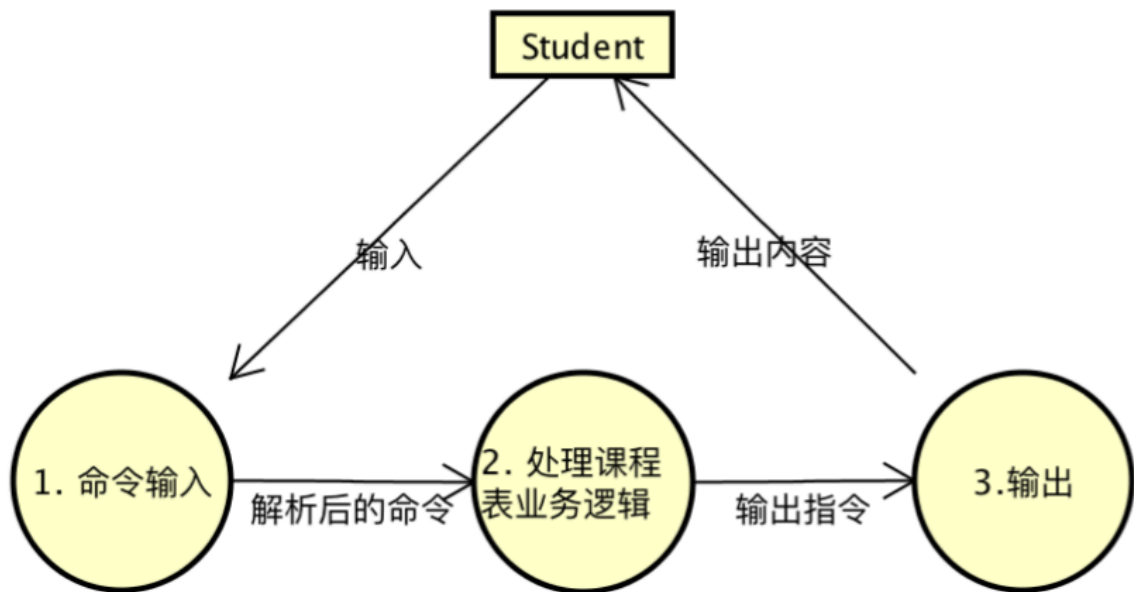


## 子过程

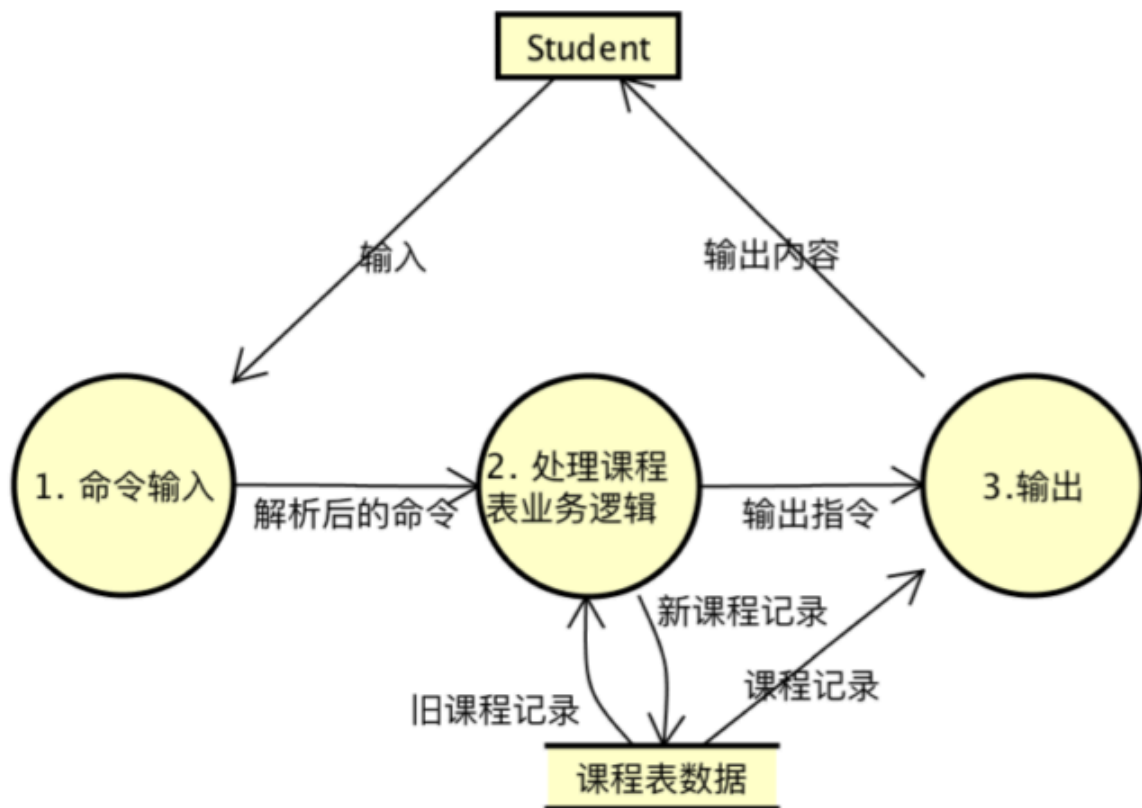


## 边界

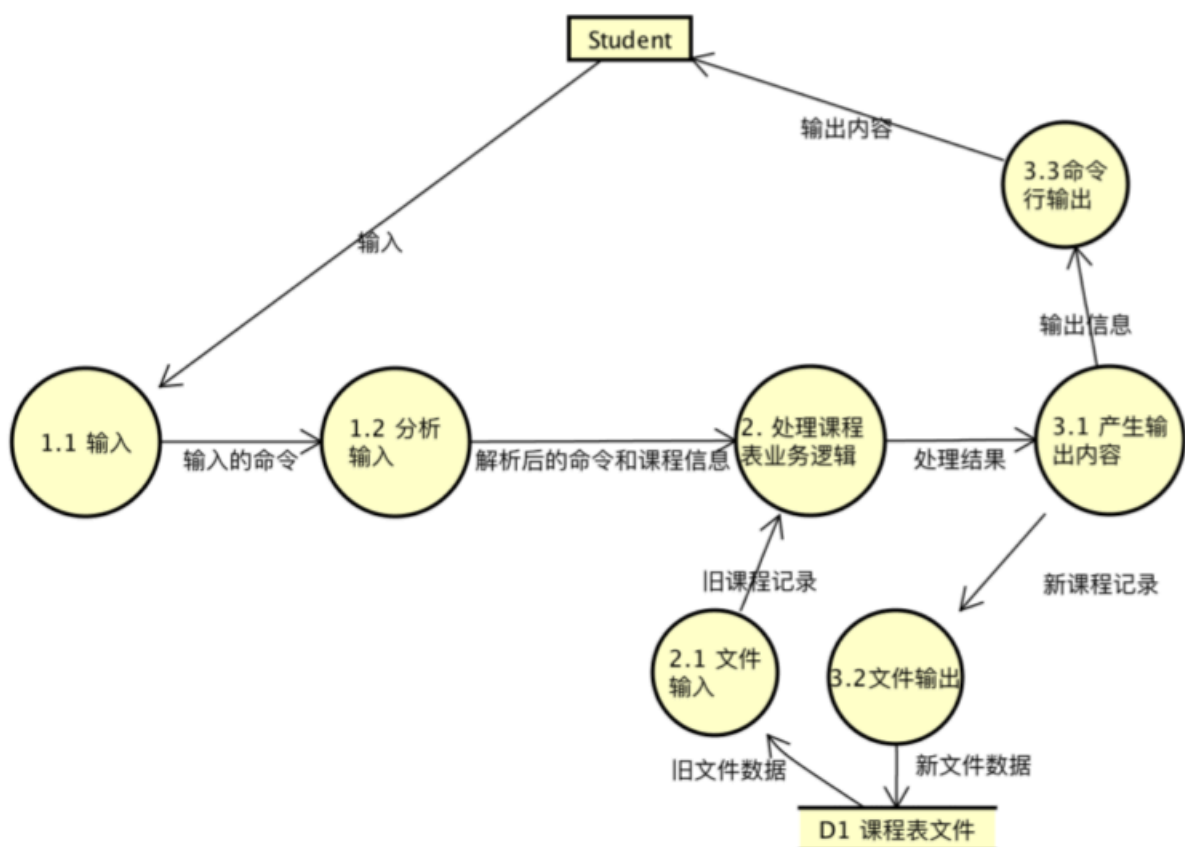
案例二——课程表



## 数据流图 - 概括



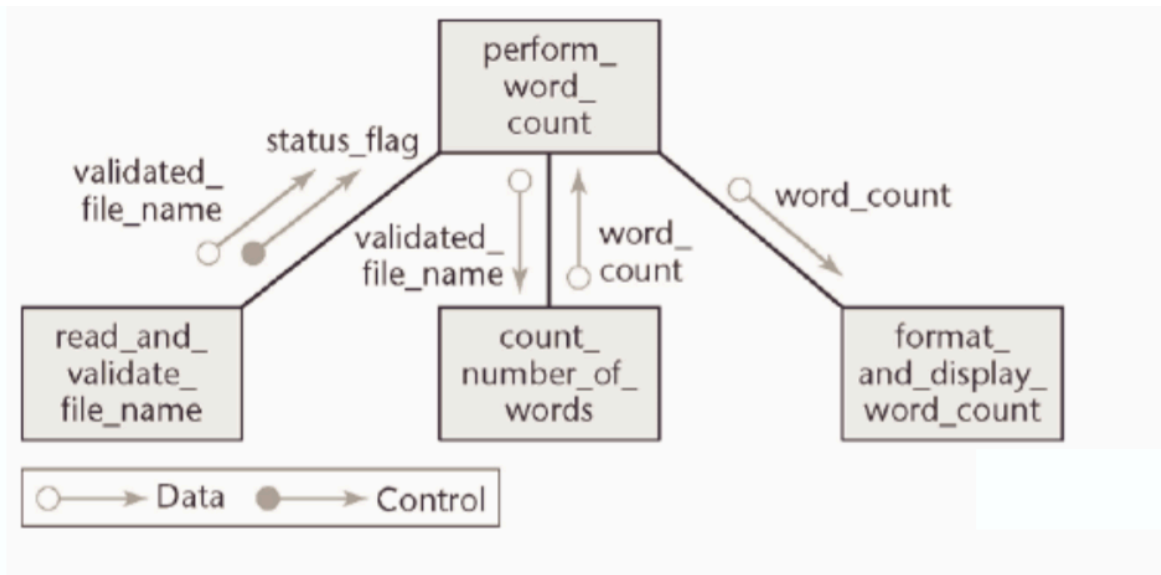
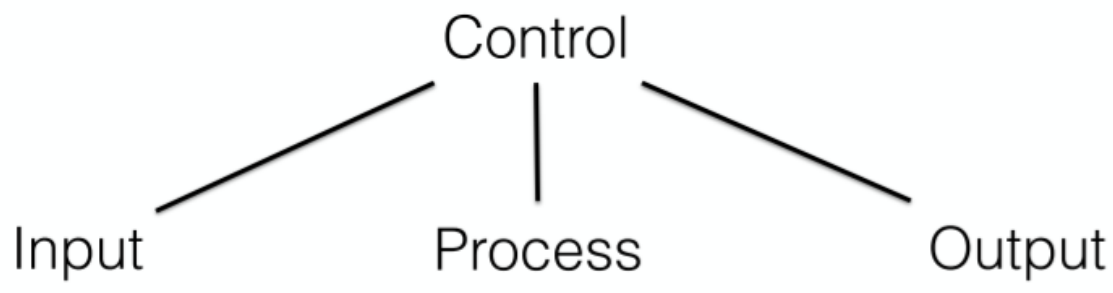
## 数据流图 - 细化



## 数据流图 - 子过程

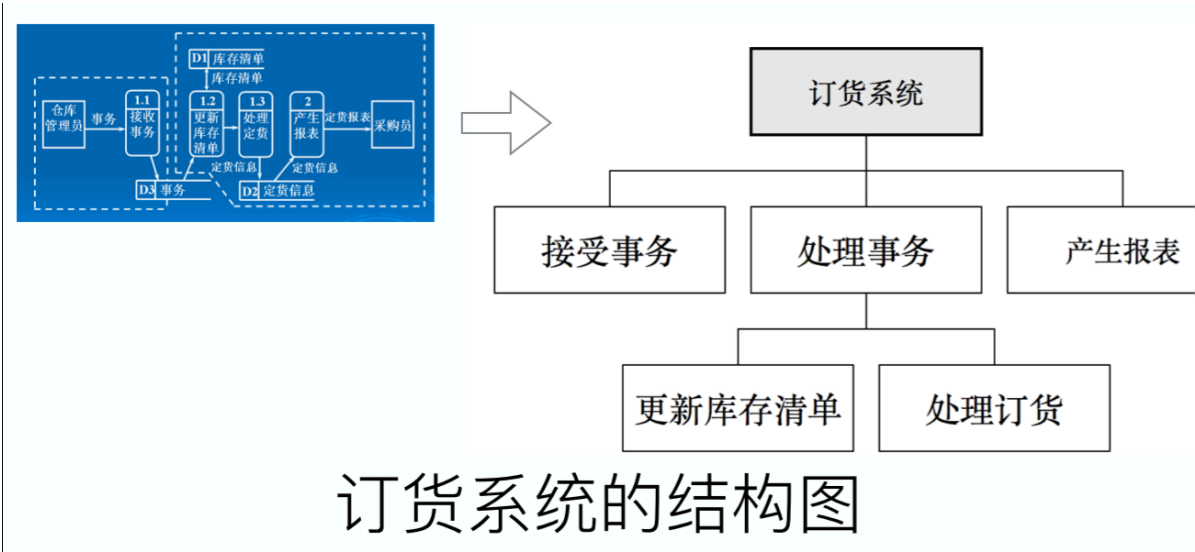
3、结构图（均使用方框和短线，短线上标注数据的流动）

主要结构为最上层的**控制中心**，两边的**处理所有的输入和所有的输出**和**中间的处理过程**（3个里面需要具体再细分）：

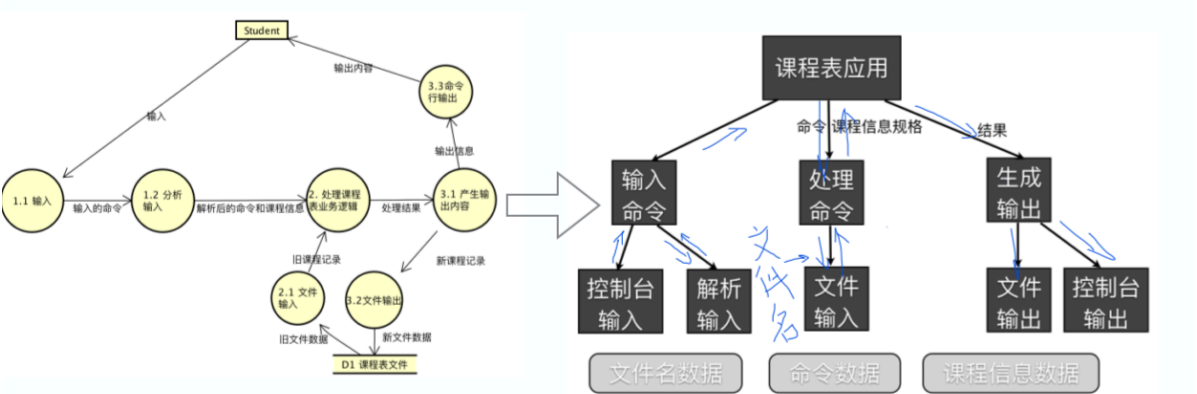


案例

案例——订货系统

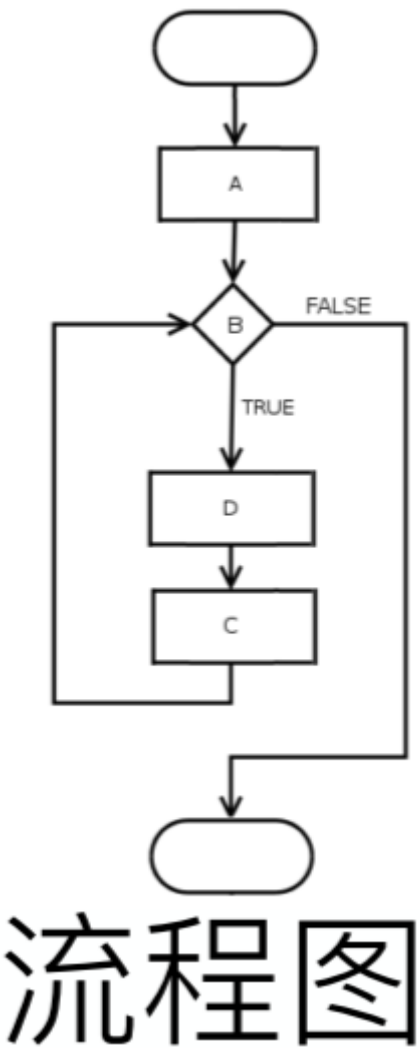


案例二——课程表



课程表结构图

4、流程图



流程图

5、结构化编程总结

根据行为来分解过程，接着设计数据来配合行为过程

## 七、结构化编程II-变量

---

### 1、变量的概念

- 物理的角度：计算机的数据存储单元
- 逻辑的角度：软件模型中的抽象数据单元
- 语义的角度：类的属性、方法的状态（每一个变量的不同的值都表示方法的不同状态）
- 优点：通过赋值语句，打破了引用透明性，能够以一种更模块化的方式构造系统（即同一个东西可以相互替换，如 $a=a+1$ ，其中两个 $a$ 的值不同）

（函数的返回值只依赖于其输入值，这种特性就称为引用透明性，如数学中 $\sin x$ 中， $\sin x$ 的值只取决于 $x$ ，没有副作用，而在程序中，函数的执行不仅依赖于输入值，而且会受到全局变量，输入文件，类的成员变量等诸多因素的影响）

- 缺陷：在命令式编程中，一个变量的值是不定的，所以**赋值的顺序**很重要，顺序的改变，会导致不同的结果。而函数式编程中，每个符号都是一个确定的值，所以顺序其实关系不大，对于每一个值都有相应的符号/变量

### 2、变量的属性

- **引用类型在堆里，基本类型在栈里。**
- 相比于 $\text{float}$ ， $\text{double}$ 更为精确，因此， $a * (b/a) \neq b$ 的情况更少出现，同时两者的误差页更小
- 均采用第一个单词首字母小写，其后大写的方式，常量应该全部大写，单词间用下划线分隔
- 因此变量的生存期开始于将变量绑定到一个特定存储单元的时刻，而结束于该变量从这个存储单元上解除绑定之时。

根据生存期将它们分成四种类型，分别被称为**静态变量**、**栈动态变量**、**显性堆动态变量**和**隐性堆动态变量**。

- 如果一个变量声明于一个子程序单元或程序块之内，那么它就是这个子程序单元或程序块内的**局部变量**。程序单元或程序块的**非局部变量**是指不在这个程序单元或程序块中声明、但又对其可见的变量。

### 3、变量的行为

- 构成声明语句
- 和操作符构成表达式
- 初始化

### 4、常用变量用法

- 局部变量
- 属性字段（作为 $\text{flag}$ 或者 $\text{state}$ 等）

## 八，结构化编程II-方法

---

### 1、方法的概念

#### 1. 概念：

- 物理角度：指令块
- 逻辑角度：抽象指令单元
- 语义的角度：行为
- 分类：静态方法（类方法），成员方法

#### 2. 方法的属性：

- 名字
- 所有者
- 接口：方法名，返回值，参数（个数，顺序，类型），**抛出的异常**
- 实现
- 运行期
- 可见性

### 2、方法的实现

#### 1. 方法的声明

#### 2. 方法的调用：

- **同步**：主函数等待调用方法执行完毕再继续执行，用实心箭头表示；**异步**：主函数不等待调用的方法，用鱼骨箭头表示
- A method uses **parameters**. A caller passes **arguments**.

**parameter=形参(formal parameter)即方法声明中的参数，argument=实参(actual parameter)即调用者传入到方法中的参数**

#### 3. 方法的结构

### 3、方法的常见用法

方法重载overloading

（在一个类中，不同的方法恰好名字相同）

重载的方法必须改变参数类型：**个数不同、类型不同、个数和类型相同但顺序不同**（但要注意，两个相同类型的交换顺序是无效的，如int a, int b)

## 面向对象编程 I -思想

---

### 面向对象思想

以前是函数之间的调用，现在是有职责的对象之间的交互

- 行为视角——结构化方法
- 数据视角——数据为中心方法
- 职责视角——面向对象方法

#### 1. 职责：

数据职责和行为职责

所谓职责，我们可以理解它为功能，每个类应当只有单一职责

## 2. 协作

## 创建类的原因

- 对现实世界中的对象建模
- 对抽象对象建模
- 降低复杂度
- 隔离复杂度
- 隐藏实现细节
- 限制变化所影响的范围
- 创建中心控制点

## 面向对象编程III-继承和多态

---

### 继承

定义：子类继承了父类所有的成员变量和方法

- 子类继承了父类所有的成员变量和成员方法
- 可以增加成员变量和成员方法
- 可以覆盖父类的成员方法
- 不可以覆盖父类的成员变量

### 多态

定义：多态是同一个行为具有多个不同表现形式或形态的能力。多态就是同一个接口，使用不同的实例而执行不同操作。

条件：继承、重写、基类引用指向派生类对象

局限：**不能使用派生类特有的成员属性和派生类特有的成员方法。**

## UML类图

---

### 可见性：

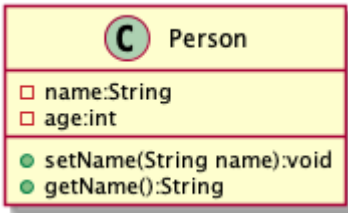
- +：表示public
- -：表示private
- #：表示protected

属性的完整表示方式是这样的：（[]内为可选）

可见性 名称：类型 [= 缺省值]

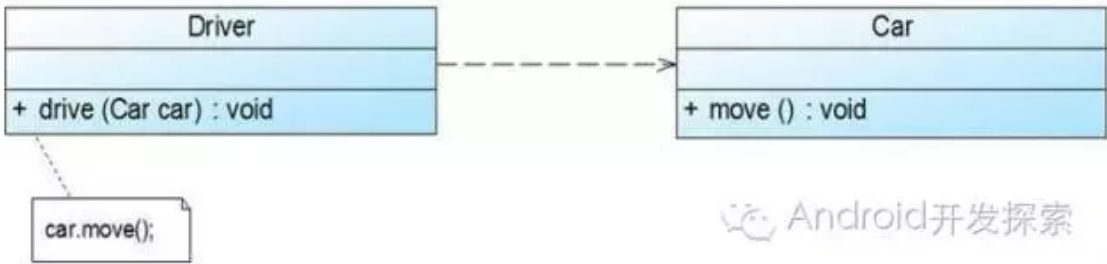
方法的完整表示方式如下：

可见性 名称(参数列表) [： 返回类型]



类与类的关系：

1. 依赖：用虚线箭头

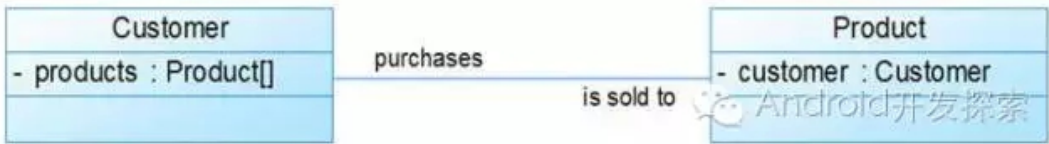


2. 关联：

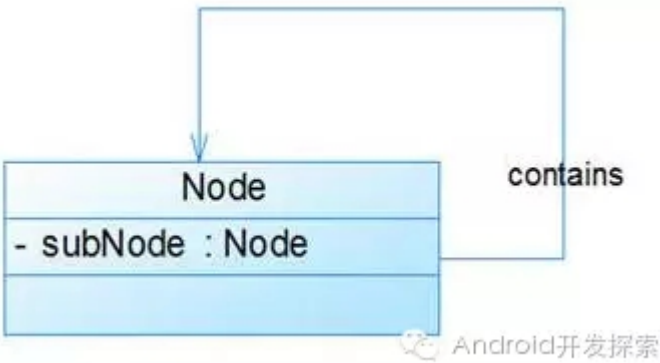
◦ 单项关联：用实线箭头



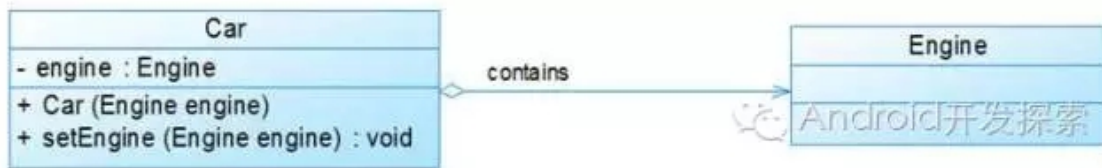
◦ 双向关联：不带箭头的实线



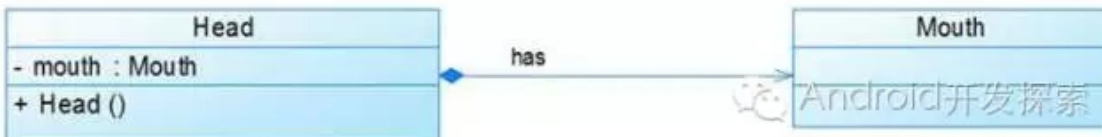
◦ 自关联：实线箭头指向自己



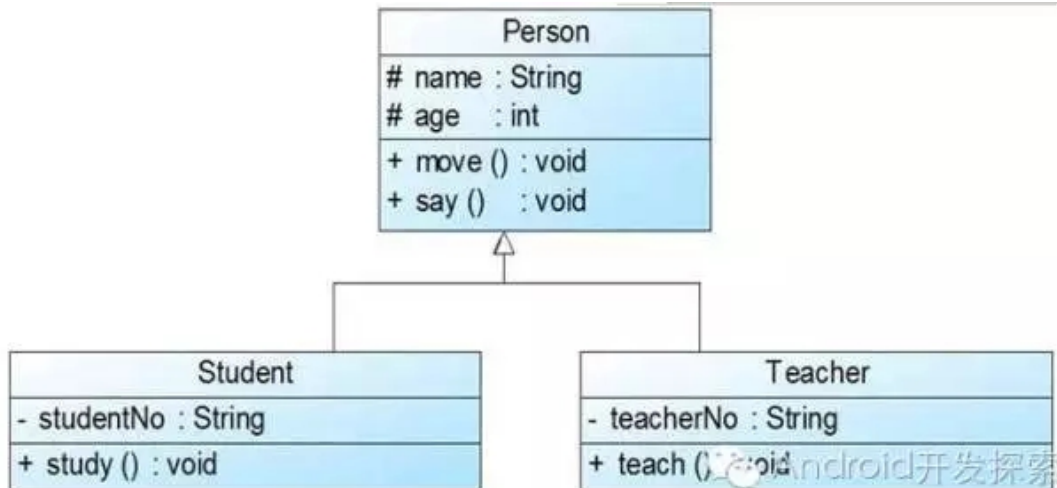
3. 聚合：带空心菱形的实线箭头



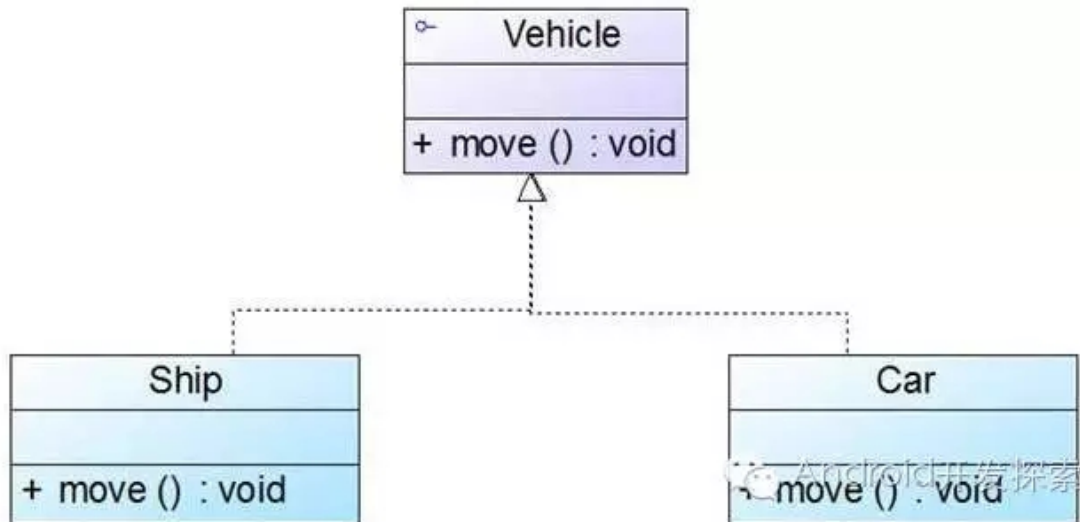
4. 组合：带实心菱形的实线箭头



5. 继承：带三角空心箭头的实线



6. 实现：带空心三角箭头的虚线



## UML用例图

# GUI

---

## GUI的三个关键要素

- 组件
- 布局
- 事件

读写均要有try catch，会存在IOException的异常

别忘了frame的尺寸和可见性

别忘了textArea的三个属性设置

```
1 | textArea.setLineWrap(true);  
2 | textArea.setWrapStyleWord(true);  
3 | textArea.setEditable(false);
```